

----- VECTOR

- Cuando se conoce la cantidad total de elementos que se tienen que almacenar siempre se debe utilizar una estructura de datos estática para su almacenamiento.
- Cuando se conoce la cant máxima de elementos que se tienen que almacenar siempre se debe utilizar una estructura de datos estática (vector/matriz) para almacenar dichos elementos..
- Si conozco la cantidad de elementos máxima que van a ser almacenados en una estructura siempre es más eficiente que esa estructura sea un vector.
- Si se conoce la cantidad máxima de elementos a almacenar, el vector es siempre la estructura de datos más adecuada para que la solución tenga el menor tiempo de ejecución.

F → Aunque un vector permite acceso directo y suele ser rápido, no es obligatorio, y no siempre es lo más eficiente en tiempo. Si hay muchas inserciones/eliminaciones intermedias, una lista puede ser más eficiente.

- Siempre es posible realizar la eliminación de un elemento en un vector.

F → No si el vector está vacío

- El acceso a un elemento de una estructura de datos lineal sólo es posible a través de un recorrido secuencial.

F → En un vector (estructura lineal) hay acceso directo por índice; en una lista sí es secuencial.

- Un vector se utiliza teniendo en cuenta siempre la dimensión lógica

V → Se declara con una dimensión física fija, pero se trabaja con una dimensión lógica (cantidad realmente usada).

Un **vector** se declara con una **dimensión física fija** (ejemplo: `array[1..1000] of integer`), es decir, reserva un espacio máximo en memoria.

Pero en la práctica, rara vez se llenan todos los espacios. Por eso se maneja una **dimensión lógica**, que indica cuántos elementos del vector están realmente ocupados. Esto permite: Evitar recorrer posiciones vacías, controlar que no se inserten elementos más allá de la capacidad máxima, Manejar dinámicamente la cantidad de datos cargados dentro de un límite fijo.

- Un vector pasado por referencia a un módulo ocupa más memoria que una lista pasada por valor. Ambas estructuras tienen la misma cantidad de elementos

F → Una referencia ocupa 4 bytes por lo tanto una referencia a un vector de cualquier tipo ocupa 4 bytes, la lista es un puntero a un nodo, por lo tanto también ocupa 4 bytes.

Entonces para la memoria estática es lo mismo (porque ambos son punteros).

Respecto a memoria dinámica siempre es más eficiente un vector porque tiene la memoria contigua y cada elemento no almacena una referencia al próximo, como la lista si lo hace.

----- MATRIZ

- Si tengo que representar los datos de lluvias caídas en un mes del año conviene utilizar una matriz frente a un vector.

F → Para registrar las lluvias de un mes del año (30 o 31 días), alcanza con un vector, ya que solo necesito guardar una lluvia por día.

Una matriz sería necesaria si, por ej, se quiere guardar las lluvias de varios meses, .

----- LISTA

- En un módulo que agrega un elemento al final de una lista, es correcto recibir el puntero inicial de la lista como un parámetro por valor.

F → Si se recibe por valor y se modifica el puntero inicial, los cambios no afectan fuera del módulo. Para modificar la lista real se debe recibir por referencia.

- Agregar un elemento al final de una lista es menos eficiente en tiempo de ejecución que agregar un elemento en un arreglo.

Depende: en un arreglo (sin espacio libre) puede requerir desplazar elementos; en lista, insertar al final con puntero al último es $O(1)$.

- Para modificar los valores contenidos en una lista se puede utilizar un módulo que sea una función.

V → Pascal permite funciones con parámetros por referencia o que devuelvan un puntero a nodo; aunque por estilo muchas veces se hace con procedimientos.

- Si p es una variable de tipo puntero, entonces siempre es válida la siguiente instrucción: `writeln(p^)`.

La instrucción `writeln(p^)` solo es válida cuando p apunta a una dirección válida. Eso implica:

- p no debe ser `nil`. error en tiempo de ejecución (acceso a puntero nulo).
- La memoria a la que apunta p debe estar correctamente asignada y no liberada.
no hacer `dispose(p)` y luego `writeln(p^)`,
- p debe haber sido inicializada. Un puntero no inicializado puede contener basura; por eso es buena práctica inicializarlo a `nil` al declararlo (`p := nil;`) o asignarlo con `new`.
- Tipo compatible con de `writeln` para el tipo apuntado.
 - $p^$ desreferencia el puntero y devuelve el valor (o registro) apuntado.
 - `writeln(p^)` funciona si el tipo apuntado es un tipo que `writeln` puede imprimir (por ej: `integer`, `real`, `string`)
- Si p es una variable de tipo puntero, entonces no es válida la siguiente instrucción: `readln(p^)`.

F → La instrucción `readln(p^)` sí es válida, si p apunta a una dirección de memoria válida: p no es `nil`, y la memoria fue reservada previamente con `new(p)` (o apunta a una variable existente correctamente inicializada).

- Antes de reservar memoria para una variable del tipo puntero, siempre se la debe inicializar en `nil`.

F → Es una buena práctica, no obligatoria, pero evita errores si luego se verifica `if p <> nil then ....` Así se evita liberar o usar memoria no reservada.

- Nunca es posible acceder al nodo de la posición N en una estructura del tipo lista.

F → Sí, es posible recorriendo secuencialmente hasta la posición N .

----- MEMORIA

- Corrección y eficiencia representan el mismo concepto.

F – Corrección significa que el programa hace lo que debe, resuelve el problema; eficiencia es hacerlo optimizando tiempo/memoria.

- Los parámetros por valor enviados a un procedimiento siempre ocupan más memoria que los parámetros enviados por referencia.

F – Los parámetros por referencia no ocupan memoria.

- Un arreglo de enteros con dimensión lógica igual a 250, es más eficiente en cuanto a memoria que una lista con 250 nodos que almacenan un entero.

V – Una lista requiere punteros extra, lo que aumenta la memoria usada, el arreglo es contiguo y solo almacena los enteros

Vector: 250 enteros * 4 bytes = **1000 bytes aprox.**

Lista: cada nodo = entero (4 bytes) + puntero (4 bytes) = 8 bytes → 250 * 8 = **2000 bytes aprox.** La lista usa el doble de memoria.

- La técnica de debugging puede aplicarse en cualquier instancia del desarrollo de un programa.

V – Se puede depurar en cualquier etapa, no solo al final.

- En la técnica de debugging es necesario analizar los casos límite del problema.

V - Justamente, los errores suelen aparecer en casos extremos.

----- GENERALES

- Las acciones dentro de una estructura de control **for** siempre se ejecutan al menos una vez.

F – por ej for:= 1 to dimL do write (v[i]); → si la dimL es 0 (vector vacío) (no se cumple la condición inicial)

- Si en un programa se encuentra la estructura de control IF siempre puede ser reemplazada por un CASE.

F – CASE solo sirve para comparar una misma variable/expresión discreta contra valores concretos, no para condiciones generales.

Ejemplo que NO puede reemplazarse con CASE:

```
if (edad > 18) and (trabaja = true) then  writeln('Es mayor de edad y trabaja')
else  writeln('No cumple las condiciones');
```

No se puede escribir con **CASE** porque intervienen expresiones booleanas y operadores lógicos (>, and), no un único valor discreto.

- Un tipo de dato registro puede ser homogéneo si todos sus campos son del mismo tipo de dato.

V → Es homogéneo si todos los campos comparten tipo.

- La operación de pertenencia de conjuntos en Pascal requiere de dos conjuntos como operandos.

La operación de pertenencia usa el operador **in** y un elemento individual a la izquierda, no un conjunto: **if 'a' in vocales then writeln('Es vocal');** 'a' es un elemento, no un conjunto.

- La dimensión lógica de un arreglo permite conocer la cantidad de memoria reservada por esa estructura.

F → La dimensión lógica indica cuántos elementos están cargados efectivamente, no cuánta memoria ocupa. La memoria reservada depende de la dimensión física

- Un tipo de dato registro es una estructura de datos de acceso secuencial.

F → Un registro (record) agrupa campos heterogéneos (distintos tipos) que se acceden por nombre, no secuencialmente. El acceso es directo, no secuencial como en una lista.

- En la técnica de corrección de debugging es necesario analizar los casos límites del problema

F → Los casos límite se analizan en el testing

----- MÓDULO

- Un módulo procedimiento no puede contener la declaración de tipos de datos (type).

F – Sí puede declarar tipos dentro de su bloque, aunque el alcance será local al procedimiento.

- Un módulo **función** no puede retornar un tipo de dato puntero a un arreglo.
- Un módulo función no puede retornar un tipo de dato definido por el usuario.
- Un módulo función puede retornar los siguientes tipos de datos: integer, boolean, char, puntero, string, real.

F → las funciones solo devuelven valores de tipo simple (int, bool, char).

- No es posible la utilización de las variables globales para la comunicación entre los módulos de un programa.

F – Es posible, aunque no siempre recomendable.

- Un programa modularizado puede no ser correcto.

V – Modularización no garantiza corrección; solo mejora estructura y mantenimiento.

- Si un programa utiliza variables globales entonces no puede contener módulos que utilicen parámetros.

F → No hay incompatibilidad, puede tener ambas cosas.

- Un módulo con 5 líneas de código es más eficiente en tiempo de ejecución que un programa con 100 líneas de código.

F → La eficiencia depende del algoritmo, no de la cantidad de líneas.

- La modularización siempre nos permite implementar programas con un tiempo de ejecución óptimo

F → La modularización mejora la organización, legibilidad y mantenimiento del código, pero no garantiza eficiencia en tiempo.

Un módulo sigue utilizando tiempo de ejecución

- Un programa que utiliza solo variables globales no requiere modularización

F – Modularizar sigue siendo útil para dividir responsabilidades, legibilidad y mantenimiento.

- Un módulo procedimiento no puede contener la declaración de tipos de datos (type).

En Pascal sí puede declararse un *type* dentro de un procedimiento o función; el tipo será local a ese módulo.

`procedure ejemplo (x);`

`type cadena = string[20];`

`var s: cadena;`

`begin s := 'Hola'; writeln(s); end;`